Arquitetura e Avaliação de um Resolvedor para ARC-AGI-1 com LLMs: Fim a Fim e Geração de Código com Laço de Correção (Nível 2)

Mike Carvalho Borges
Especialização em Inteligência Artificial Avançada, UFRGS
mike.borges@ufrgs.br

Resumo

Este trabalho apresenta o projeto, a implementação e a avaliação empírica de um resolvedor didático para o benchmark ARC-AGI-1, em Python e gerenciado via uv. O sistema emprega o Gemini 3.5 Flash Lite com raciocínio estendido (high thinking) e implementa duas abordagens autônomas: inferência matricial direta (Fim a Fim) e síntese de programas em Python (Geração de Código) em sandbox isolado com controle de tempo limite. O módulo de código integra um laço de correção reflexivo baseado em diagnósticos de erro nos pares de treino. Em 20 tarefas representativas, o resolvedor obteve 85% de acurácia (17/20) em cada abordagem (90% na união). O laço de correção recuperou 3 tarefas inicialmente incorretas (média de 1,40 tentativas/tarefa), e o ambiente identificou 2 casos de sobreajuste indutivo que acertaram 100% do treino e falharam no teste.

1 Introdução

O ARC-AGI-1 (Abstraction and Reasoning Corpus) foi proposto por François Chollet em 2019 como benchmark de inteligência fluida e eficiência amostral. Ao contrário de tarefas de classificação supervisionada, o ARC exige induzir regras de transformação espacial a partir de apenas 2 a 5 pares de treino de matrizes coloridas (valores 0 a 9) e aplicá-las a uma grade de teste inédita.

Para analisar a capacidade de modelos de fronteira com raciocínio estendido, construímos um resolvedor modular em torno do Gemini 3.5 Flash Lite (gemini-3.5-flash-lite, thinking high), comparando duas estratégias:

1. Fim a Fim (End-to-End): Inferência direta em que o LLM raciocina e emite a matriz de teste serializada em JSON.
2. Geração de Código (Code Generation): Síntese de uma função pura transform(grid) executada em sandbox e aprimorada via laço iterativo de correção reflexiva.

2 Arquitetura e Método do Resolvedor

O resolvedor foi estruturado em um único ponto de acesso (solver.py), sob o gerenciador ultrarrápido uv.

2.1 Fluxo de Dados e Representação Matricial

O resolvedor lê o arquivo tarefas.txt e processa as tarefas em JSON. Cada grade é formatada com dimensões explícitas ($L \times C$), formatação visual tabular e legenda de cores (0: Preto a 9: Marrom), facilitando o raciocínio geométrico e topológico do modelo.

2.2 Módulo Fim a Fim

No modo Fim a Fim, o modelo deduz as invariâncias e retorna um objeto JSON com a chave "output". O analisador sintático combina busca por blocos "'json, inspeção de dicionários e expressões regulares de salvaguarda.

2.3 Módulo de Código e Sandbox Seguro

No modo de código, o LLM sintetiza a função pura def transform(grid: list[list[int]]) -> list[list[int]]:. A função é executada em processo isolado (subprocess) com:

- Tempo Limite (Timeout): Interrupção forçada após 5,0s por grade para neutralizar laços infinitos.
- Tratamento de Exceções: Captura segura de IndexError, erros de recursão e tipos sem interromper o resolvedor.

2.4 Laço de Correção Reflexivo

O diferencial do método é o laço de correção (*reflection loop*). Quando o código sintetizado falha em exemplos de treino:

1. O sandbox intercepta saídas incorretas ou erros por par de treino.
2. Constrói-se um prompt de diagnóstico com o código prévio, os contraexemplos (grade esperada vs. obtida) e a lista dos pares que passaram (preservando funcionalidades válidas).
3. O modelo reflete sobre a falha e gera uma versão corrigida (limite configurável de até 3 tentativas).
4. Ao atingir 100% de acerto no treino, o laço é encerrado e a função é aplicada ao teste.

Abordagem	Acurácia	Laço	Tent.	Tr/Te	Racioc.	Tempo (s)
Fim a fim	17/20 (85%)	–	1,00	0	157.963	515,7
Código	17/20 (85%)	3	1,40	2	370.559	1.464,6

Tabela 1: Métricas consolidadas nas 20 tarefas (Gemini 3.5 Flash Lite, Raciocínio Alto). Laço: tarefas resolvidas c/ laço. Tent.: média de tentativas/tarefa. Tr/Te: casos com 100% no treino e falha no teste.

2.5 Protocolo Experimental

O resolvedor respeita 1,0s de pausa entre requisições à API e audita tokens de entrada, raciocínio interno, saída, latência da API, tempo de sandbox, número de tentativas e nota binária estrita (1,0 ou 0,0).

3 Resultados e Discussão

A Tabela 1 consolida o desempenho obtido nas 20 tarefas avaliadas (10 de treino e 10 de avaliação). Ambas as abordagens atingiram 85,0% de acurácia individual (17/20), com 100% de acerto (10/10) nas tarefas de treino (training/).

3.1 Eficácia do Laço de Correção

O laço de correção foi fundamental para o módulo de código. Em 3 tarefas, o código inicial falhou no treino, mas o laço diagnosticou as inconsistências e encontrou a solução correta no teste:

- training/44d8ac46.json: convergiu na 3ª tentativa após retificar a lógica de expansão geométrica.
- training/0a938d79.json: convergiu na 2ª tentativa, corrigindo o mapeamento de cores (4/4 no treino e nota 1,0 no teste).
- evaluation/7c9b52a0.json: convergiu na 3ª tentativa, ajustando o recorte dimensional e obtendo nota 1,0 no teste inédito.

A média foi de 1,40 tentativas por tarefa. O laço elevou o consumo de tokens de raciocínio para 370.559 (aumento de 134,6% frente ao modo fim a fim) e o tempo para 1.464,6s, refletindo o custo deliberativo adicional sobre contraexemplos.

3.2 Sobreajuste ao Treino e Falha no Teste (Tr/Te)

O resolvedor identificou 2 casos em que o código alcançou 100% de acerto no treino (3/3), mas errou o teste (Train Pass / Test Fail):

- evaluation/d47aa2ff.json: o código ajustou-se perfeitamente aos 3 pares de treino na 1ª tentativa, mas induziu critério espúrio que falhou no teste.
- evaluation/f5aa3634.json: a 1ª tentativa falhou (0/3); o laço corrigiu o código na 2ª tentativa (3/3 no treino), mas a regra ainda continha viés incompleto para o teste.

Esse resultado expõe a lacuna de generalização indutiva em aprendizado com poucos exemplos (*few-shot*), onde múltiplas hipóteses satisfazem o treino mas apenas uma reflete o teste.

3.3 Complementaridade e União dos Módulos

Houve clara complementaridade entre as abordagens:

- evaluation/aab50785.json: resolvida somente por Código (1,0). A tarefa requer extrair o interior de blocos 2×2; o fim a fim falhou por desalinhamento, enquanto o código calculou as coordenadas exatas.
- evaluation/93b4f4b3.json: resolvida somente por Fim a Fim (1,0). A tarefa envolve reflexão e simetria global, intuitiva visualmente para o LLM, mas difícil de parametrizar em código (esgotou as 3 tentativas).

A união das abordagens alcançou 18/20 tarefas resolvidas (90,0%), evidenciando os ganhos de um sistema híbrido.

4 Conclusão

O resolvedor valida o potencial do Gemini 3.5 Flash Lite com raciocínio estendido no ARC-AGI-1. O laço de reflexão recuperou tarefas críticas ao inspecionar diagnósticos no sandbox, ao mesmo tempo em que revelou casos de sobreajuste aos exemplos de demonstração. A combinação entre síntese determinística de código e intuição espacial direta constitui o caminho mais promissor para raciocínio abstrato robusto.

Uso de Inteligência Artificial: O assistente de IA foi utilizado como par de programação para a arquitetura modular do resolvedor, laço de reflexão, sandbox e automação experimental com uv.

Referências

[1] F. Chollet. On the measure of intelligence. arXiv:1911.01547, 2019.

[2] Gemini Team. Gemini: A family of highly capable multimodal models. arXiv:2312.11805, 2025.